Start here

Databricks pipeline engineer

A0-L1 Spark DataFrames, the model and the core methods

Chain transformations and use their results, let Spark aggregate, and run an action only when you need the answer.

```python
from pyspark.sql import functions as F

df = spark.table("research_prod.research_silver.nat_respondents")
```

Transformation against action	Transformations (select, filter, withColumn, groupBy, join) describe a new DataFrame. Actions (show, count, collect, write) run the plan.
Why df.withColumn(...) on its own line does nothing	DataFrames are immutable. Use the result: df = df.withColumn(...).
Summarise per group	df.groupBy("region").agg(F.sum("w").alias("total"), F.count("*").alias("n")).
How to import Spark functions	from pyspark.sql import functions as F. Never import sum, max or min from it by name.

A0-L2 The SQL that pipelines use

Filter rows with WHERE and groups with HAVING, test NULL with IS NULL, and keep one row per key with ROW_NUMBER, not DISTINCT.

```sql
SELECT respondent_id, wave_id, final_weight
FROM (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
  delivery_version DESC) AS rn
  FROM bronze_deliveries
)
WHERE rn = 1
```

WHERE against HAVING	WHERE filters rows before grouping. HAVING filters groups after aggregation.
COUNT(*) against COUNT(column)	COUNT(*) counts rows. COUNT(column) counts non-NULL values. SUM and AVG skip NULLs.
Find rows with no match in a lookup	LEFT ANTI JOIN lookup ON key. A LEFT JOIN keeps all rows and fills NULLs.
Test for a missing value in SQL	IS NULL or IS NOT NULL. = NULL is never true.
Keep the latest row per key	ROW_NUMBER() OVER (PARTITION BY keys ORDER BY version DESC) AS rn, then WHERE rn = 1.

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers

Name every table in full, keep files in volumes, let Delta keep versions, and move data forward from bronze to silver to gold.

Unity Catalog name	catalog.schema.table, such as research_prod.research_silver.nat_respondents. Files live in volumes.

What makes a Delta table	Parquet data files plus _delta_log, where each write adds a numbered commit: versions 0, 1, 2.
Read an older version of a table	SELECT * FROM t VERSION AS OF 1. DESCRIBE HISTORY t lists the versions.
The three medallion layers	Bronze: as received, with audit columns. Silver: clean, one row per key. Gold: aggregates with a stated grain.